

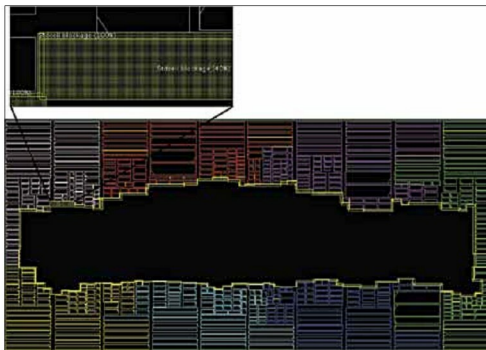


Fig. 3: Placement

Uniform core area. Another challenge is to create a uniform core area so as to provide better accessibility to memories that are stacked away from the core. The shape of the core area should neither have hillocks nor valleys. In order to achieve this milestone, we tried to control the stack size. The stack size trimming, breaking one bigger stack into two smaller stacks and rearrangement of these stacks helped in achieving a uniform core area.

Timing concerns. The access time of some of the memories is more than half of the clock period. These memories have 30 to 40 per cent contribution to FEP (front end processing) and more than 80 per cent contribution to TNS (total negative slack) if they are placed away from the core area. To avoid this timing criticality, such memories are placed near the core area. This places the related logic near it which, in turn, reduces the wire buffering and slew, and hence the slack. It has a minute impact on congestion, which can be ignored until later stages.

Power planning challenges

As mentioned earlier, the design has eight metal layers with HVH (high velocity hydroforming) process, of which the topmost layers are specifically used

for power mesh and top-level routing. The bottommost layer for power mesh is the seventh layer, which is horizontal and leads to incomplete rail structure, causing IR drop and congestion issues. To avoid this in the core area, it is necessary to place a custom mesh in six vertical metal layers. Also, in vertical channels between stacks, macro rings in Metal 4 are created.

The routing blockages in memories are till Metal 4, so M5 and M6 are used for routing over macros. Creating the M6 mesh limits these resources and causes congestion over macros. To tackle this issue, we used a window-based mesh structure and created the M6 mesh only in the uniform core region.

Placement challenges

Blockages. The design has a lot of stacks of memories, which forces the placer to place more and more cells very close to the macro and core interface, increasing congestion in those areas. To handle this, 100 per cent standard cell blockage is maintained in a very thin area of full core to memory interface.

In addition to this blockage, a 40 per cent partial placement blockage is kept in comparatively larger areas to control the cell density and avoid congestion issues. These blockages can

be seen in Fig. 1.

Also, to maintain the uniform spreading of cells throughout the core area, 10 per cent partial placement blockage throughout the core area is placed to avoid local pin density issues. This helps in mitigating congestion.

Region/cluster-based placement. Custom cell placement regions or clusters are used to optimise timing in timing-critical paths. Now as the design is highly congested and densely utilised, there are chances that related logic can be placed far apart, due to which timing violations are expected. So in order to trade-off between congestion and timing, once you are done with congestion the failing endpoints should be clustered to avoid timing violation.

Clock tree challenges

As the design contains 21 clocks, it brings along the challenge of clock tree building and handling inter-clock domains. Now as the memory is scattered very far apart, meeting skew targets becomes very difficult. In the design, we have implemented a separate tree based on logic hierarchy, wherein we have created a separate tree for six different hierarchies. By strictly maintaining skew and insertion delay targets, we have handled this issue in separate tree clock insertion delay targets. Similarly, by maintaining the insertion delay target, we tried to make sure that inter-clock domain clocks are also synchronised, which helps us achieve better clock tree structure.

To sum up

The beauty of this methodology to tackle large and complex blocks in a flat implementation is that it can be implemented or adopted by any tool or any flow. It also helps in the number of iterations and can get you closer to the desired result as large designs have very large run times and pose huge memory requirements. So with the help of this approach, you can reduce block time period and eventually the time to market. ●

The authors are from elinfochips, Ahmedabad